DL_LAB4_314553044_楊正豪
Lab 4 — Conditional VAE for Video Prediction

姓名	學號
楊正豪	314553044

A. Introduction

在 Lab 4 中，我們實作了基於（Conditional Variational Autoencoder, CVAE的 Video Prediction 模型。目標是透過給定的初始真實影像與未來的 Skeleton Pose Labels 作為條件特徵，連續生成後續的動態影像序列。

階段	機制	目的
Training	MSE Loss	確保像素級影像重建精確度
Training	KL Divergence	使動作過渡平滑
Validation	PSNR	量化生成影像畫質與預測準確度

B. Implementation Details

Reconstruction

模型首先透過 Encoder 萃取前一幀影像（x_1）與當前目標骨架（p_2）的特徵向量。接著，將這兩組條件特徵與抽樣出的潛在變數（z）共同輸入 **Decoder Fusion** 進行整合。融合後的高維特徵傳入 Generator 進行解碼，輸出下一幀的預測影像（$\hat{x}_2$）。

最後，計算預測影像與真實影像之間的重建損失：

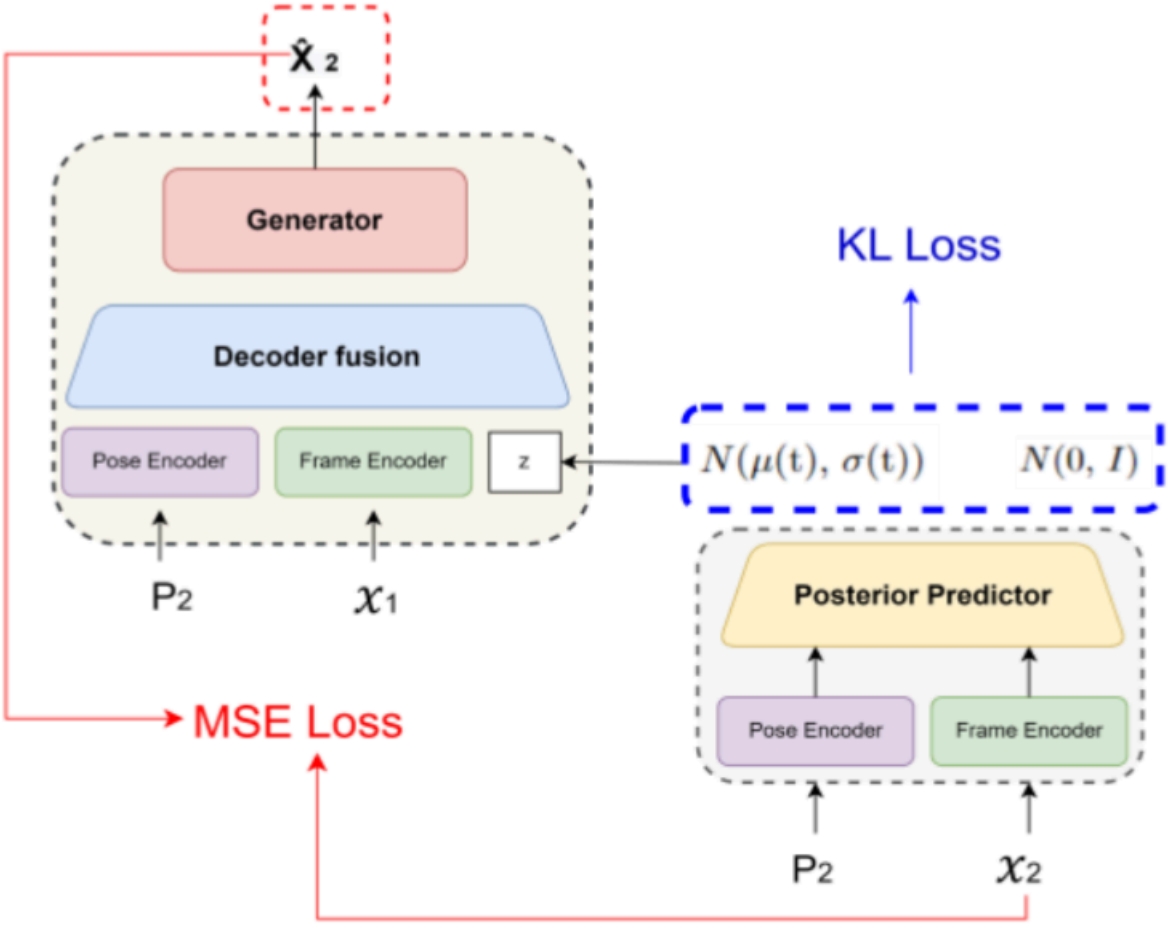
$$\mathcal{L}_{\text{rec}} = \|\hat{x}_2 - x_2\|^2$$

Posterior Approximation

模型將真實的 x_2 與 p_2 輸入至 Posterior Predictor 中(如Figure右下所示)，估測潛在變數的分佈，並從中抽樣出 z。同時計算該分佈與標準常態分佈之間的 KL Loss：

$$\mathcal{L}_{\text{KL}} = D_{\text{KL}}\big(\mathcal{N}(\mu, \sigma^2) \,\|\, \mathcal{N}(0, 1)\big)$$

Figure (1) — CVAE Architecture



---

i. Training / Testing Protocol

Training

本實驗之訓練流程基於 Figure 1 所示之架構，針對影像序列進行長度為 `sequence` 的 Autoregressive 訓練。具體步驟如下，首先初始化目標函數（MSE Loss 與 KL Loss）之累加器，並將真實序列的首幀設定為初始歷史影像 x_1（`prev_x`）：

1. **Teacher Forcing 判斷**：依據當前設定之機率決定是否觸發 Teacher Forcing。若觸發，則以上一真實幀作為 x_1；反之，則沿用模型前一次生成的預測影像作為 x_1。
2. **Posterior 抽樣與 KL Loss**：將歷史影像 x_1、當前目標影像 x_2 與骨架條件 p_2 分別輸入對應之 Encoder 萃取特徵。將 x_2 與 p_2 之特徵送入 Posterior Predictor，計算潛在空間分佈並抽樣出潛在變數 z，同時計算 KL Loss 以約束該分佈。
3. **Decoder Fusion 與影像生成**：將 x_1、p_2 以及 z 輸入至 Decoder Fusion 進行組合。融合後之特徵向量接著送入 Generator 產生當前時間步的預測影像 $\hat{x}_2$（`x2_hat`）。
4. **MSE Loss 計算**：評估預測影像 $\hat{x}_2$ 與真實影像 x_2 的均方誤差。
5. **反向傳播與參數更新**：將累積的 KL Loss 與 MSE Loss 進行加權整合後，清空梯度、執行 `backward()` 計算梯度，最後呼叫優化器更新網路權重。

```
def training_one_step(self, img, label, adapt_TeacherForcing):
    # Step 0: 初始化
    total_mse_loss = 0.0
    total_kl_loss  = 0.0
    prev_x   = img[:, 0]
    sequence = self.train_vi_len
```

```python
    for t in range(1, sequence):
        # Step 1: Teacher Forcing 判斷
        if adapt_TeacherForcing:
            x1_input = img[:, t - 1]
        else:
            x1_input = prev_x

        p2_input = label[:, t]
        x2_input = img[:, t]

        # Step 2: Posterior 抽樣與 KL Loss
        p2 = self.label_transformation(p2_input)
        x1 = self.frame_transformation(x1_input)
        x2 = self.frame_transformation(x2_input)
        z, mu, logvar = self.Gaussian_Predictor(x2, p2)
        kl_loss        = kl_criterion(mu, logvar, self.batch_size)
        total_kl_loss += kl_loss

        # Step 3: Decoder Fusion 與影像生成
        z_fusion = self.Decoder_Fusion(x1, p2, z)
        x2_hat   = self.Generator(z_fusion)
        x2_hat   = torch.sigmoid(x2_hat)

        # Step 4: MSE Loss 計算
        mse_loss        = self.mse_criterion(x2_hat, x2_input)
        total_mse_loss += mse_loss

        prev_x = x2_hat.detach()

    # Step 5: 反向傳播與參數更新
    self.optim.zero_grad()
    total_kl_loss  /= (sequence - 1)
    total_mse_loss /= (sequence - 1)
    total_loss = total_kl_loss * self.kl_annealing.get_beta() + total_mse_loss

    total_loss.backward()
    self.optimizer_step()

    return total_loss
```

Testing

對於第 t 幀影像，model 透過前一幀和當前骨架進行推論。具體方式如下：

1. 利用 Encoder 萃取前一幀預測影像 x_1 與當前骨架條件 p_2 的特徵。與 Training 不同，不需要使用 Posterior Predictor 進行採樣 z，而是以零向量作為 z，能夠有效降低 MSE loss 抽樣的期望值，穩定並提升 PSNR。
2. 將 x、p、z 送入 Decoder Fusion，再經由 Generator 輸出介於 0 和 1 之間的預測影像。
3. 將當前的預測影像 $\hat{x}_2$ 更新為下一步的輸入（x_1），反覆執行 629 次，完成整段影片的預測。

```python
def val_one_step(self, img, label, idx=0):
    # img / label: [B, seq, C, H, W] → permute to [seq, B, C, H, W]
    img   = img.permute(1, 0, 2, 3, 4)
    label = label.permute(1, 0, 2, 3, 4)
```

```python
    assert label.shape[0] == 630, "Testing pose sequence should be 630"
    assert img.shape[0] == 1,     "Testing video sequence should be 1"

    decoded_frame_list = [img[0].cpu()]
    label_list         = [label[0].cpu()]
    prev_x = img[0]

    for t in range(1, label.shape[0]):
        # Step 1 :Encoder將影像骨架轉換
        x = self.frame_transformation(prev_x)
        p = self.label_transformation(label[t])
        z = torch.zeros(x.shape[0], self.args.N_dim,
                        x.shape[2], x.shape[3]).to(prev_x.device)
       # Step 2:Decorder Fusion and Generatpr
        z_fusion = self.Decoder_Fusion(x, p, z)
        # Step 3: Update
        x2_hat   = self.Generator(z_fusion)
        x2_hat   = torch.sigmoid(x2_hat)

        decoded_frame_list.append(x2_hat.cpu())
        label_list.append(label[t].cpu())
        prev_x = x2_hat.detach()
```

---

ii. Reparameterization Trick

Reparameterization Trick 將隨機性從模型參數中分離，分三步驟完成：

1. **轉換標準差**：為數值穩定性，模型輸出 $\log \sigma^2$，再轉換為標準差：

$$\sigma = \exp\left(\frac{1}{2} \cdot \log \sigma^2\right)$$

2. **抽樣獨立雜訊**：從標準常態分佈中抽取不依賴模型參數的雜訊：

$$\epsilon \sim \mathcal{N}(0, 1)$$

3. **計算潛在變數**：透過線性組合得到最終的 z，梯度可通過 μ 和 σ 反傳：

$$z = \mu + \sigma \cdot \epsilon$$

```python
def reparameterize(self, mu, logvar):
    # Step 1: 轉換 logvar → std
    std = torch.exp(0.5 * logvar)
    # Step 2: 從標準常態分佈抽樣獨立雜訊
    eps = torch.randn_like(std)
    # Step 3: z = μ + σ·ε
    return mu + std * eps
```

---

iii. Teacher Forcing Strategy

每個 Epoch 訓練結束後呼叫 `teacher_forcing_ratio_update()`，根據當前 Epoch 動態調整 TFR：

1. **保護期**：在前 `tfr_sde` 個 Epoch 內，`current_epoch < tfr_sde` 條件不成立，TFR 維持初始值 1.0，模型全程使用真實幀作為輸入。
2. **線性衰減**：自第 `tfr_sde` 個 Epoch 起，每個 Epoch 將 `tfr` 減去固定步長 `tfr_d_step`，模型逐步轉向自回歸模式。
3. **下限保護**：透過 `max(0.0, ...)` 確保 TFR 不低於 0，防止出現負值。

```
def teacher_forcing_ratio_update(self):
    if self.current_epoch >= self.tfr_sde:
        self.tfr = max(0.0, self.tfr - self.tfr_d_step)
```

---

iv. KL Annealing Ratio

`kl_annealing` 類別在初始化時預先計算整個訓練週期的 β 排程陣列，訓練時透過 `get_beta()` 取得當前 Epoch 對應的權重：

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{MSE}} + \beta \cdot \mathcal{L}_{\text{KL}}$$

1. **初始化排程**：根據 `kl_anneal_type` 決定策略，呼叫 `frange_cycle_linear` 生成長度為 `num_epoch` 的 β 陣列：
   - **Cyclical**：`n_cycle = kl_anneal_cycle`，β 在每個週期的前 `ratio` 比例內從 0 線性爬升至 1，其餘填滿 1，下一週期重置
   - **Monotonic**：`n_cycle = 1`，等同於僅執行一次線性爬升，前 `num_epoch × ratio` 個 Epoch 從 0 增至 1，之後恆為 1
   - **None**：直接產生全為 1 的陣列，β 全程固定，不做任何退火
2. **frange_cycle_linear**：先將陣列初始化為全 1，再逐週期從 `start=0` 以步長 `step = 1 / (period × ratio)` 線性填入，達到 1 後停止，未被覆蓋的位置維持 1。
3. **取得與更新 β**：`get_beta()` 以 `current_epoch` 為索引直接取值；`update()` 在每個 Epoch 結束後將計數器加一。

```
class kl_annealing():
    def __init__(self, args, current_epoch=0):
        # Step 1:判斷是哪個函數
        self.current_epoch = current_epoch
        if args.kl_anneal_type == 'Cyclical':
            self.beta = self.frange_cycle_linear(
                args.num_epoch, 0, 1,
                n_cycle=args.kl_anneal_cycle,
                ratio=args.kl_anneal_ratio)
        elif args.kl_anneal_type == 'Monotonic':
            self.beta = self.frange_cycle_linear(
                args.num_epoch, 0, 1,
                n_cycle=1,
                ratio=args.kl_anneal_ratio)
        elif args.kl_anneal_type == 'None':
            self.beta = np.ones(args.num_epoch)
        # Step 3:取得權重
    def get_beta(self):
        return self.beta[self.current_epoch]
```

```python
def update(self):
    self.current_epoch += 1
    # Step 2 :frange_cycle_linear
def frange_cycle_linear(self, n_iter, start=0.0, stop=1.0, n_cycle=1, ratio=1):
    L      = np.ones(n_iter) * stop
    period = n_iter / n_cycle
    step   = (stop - start) / (period * ratio)
    for c in range(n_cycle):
        v, i = start, 0
        while v <= stop and (int(i + c * period) < n_iter):
            L[int(i + c * period)] = v
            v += step
            i += 1
    return L
```

C. Analysis & Discussion

i. Plot Teacher Forcing Ratio (5%)

Hyperparameters

參數	值
Epoch	200
Mode	Cyclical
Optimizer	Adam
Batch Size	4
Learning Rate	1e-4
KL Anneal Cycle	20
KL Anneal Ratio	0.5
Teacher Forcing Ratio（初始）	1.0
Teacher Forcing 開始衰減	Epoch 10
Teacher Forcing 衰減步長	0.05

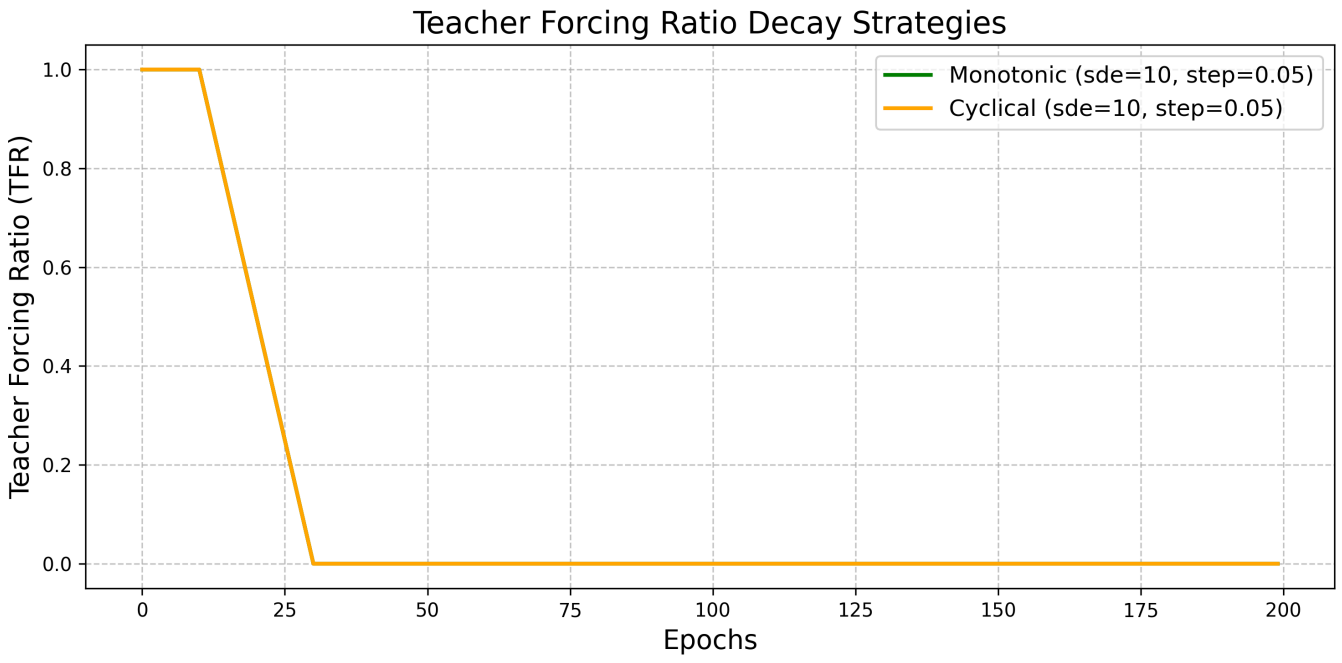


Figure (2) — Teacher Forcing Ratio Decay Strategies

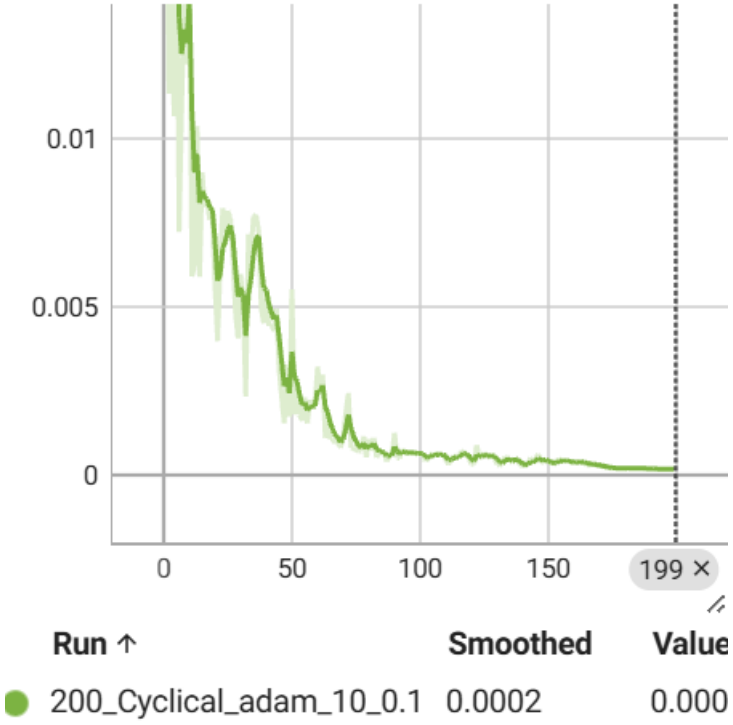


Figure (3) — MSE Loss Curve（200 Epochs, Cyclical）

Conclusion

TFR 從第 10 個 Epoch 起線性衰減，至第 30 個 Epoch 降至 0 後維持不變。在 TFR 衰減期（Epoch 10–30），MSE Loss 出現明顯的 Spike 與震盪；此後隨訓練推進逐漸趨於穩定，至 Epoch 199 時收斂至約 **0.0002**。

---

ii. Plot the Loss Curve with Different Settings (5%)

Hyperparameters

參數	值
Epoch	70
KL Annealing Mode	依曲線各異
Optimizer	Adam
Batch Size	4
Learning Rate	1e-4
KL Anneal Cycle	20
KL Anneal Ratio	0.5
Teacher Forcing Ratio（初始）	1.0
Teacher Forcing 開始衰減	Epoch 10
Teacher Forcing Ratio（衰減）	0.1

Cyclical

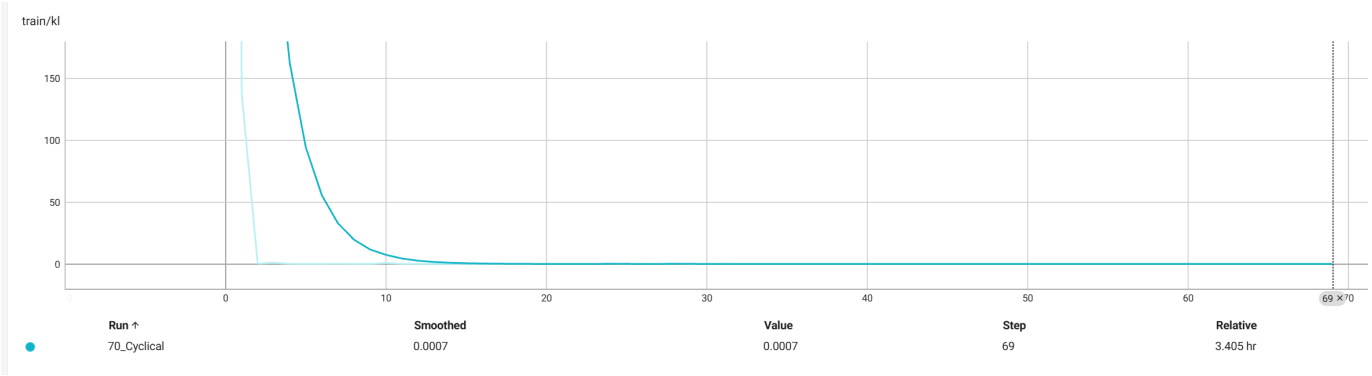


Figure (4) — Cyclical KL Loss

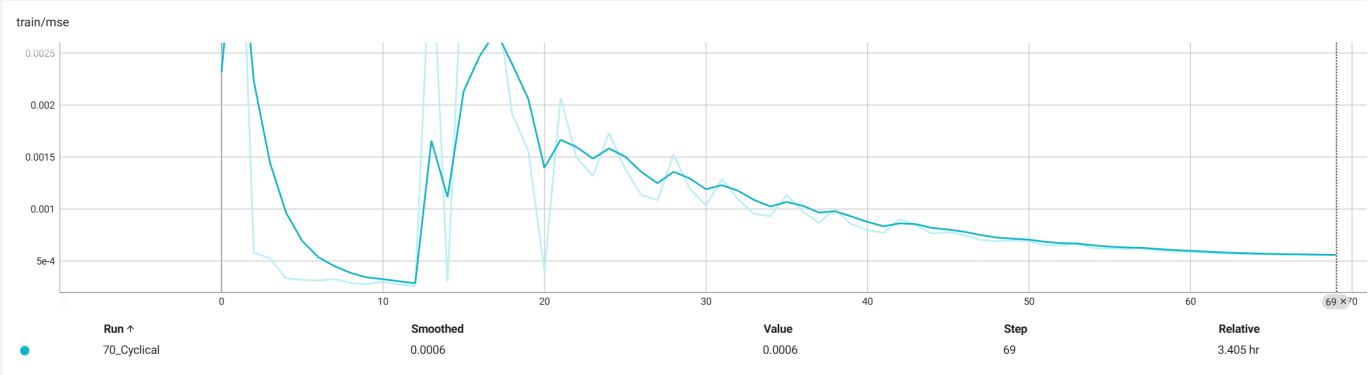


Figure (5) — Cyclical MSE Loss

Monotonic

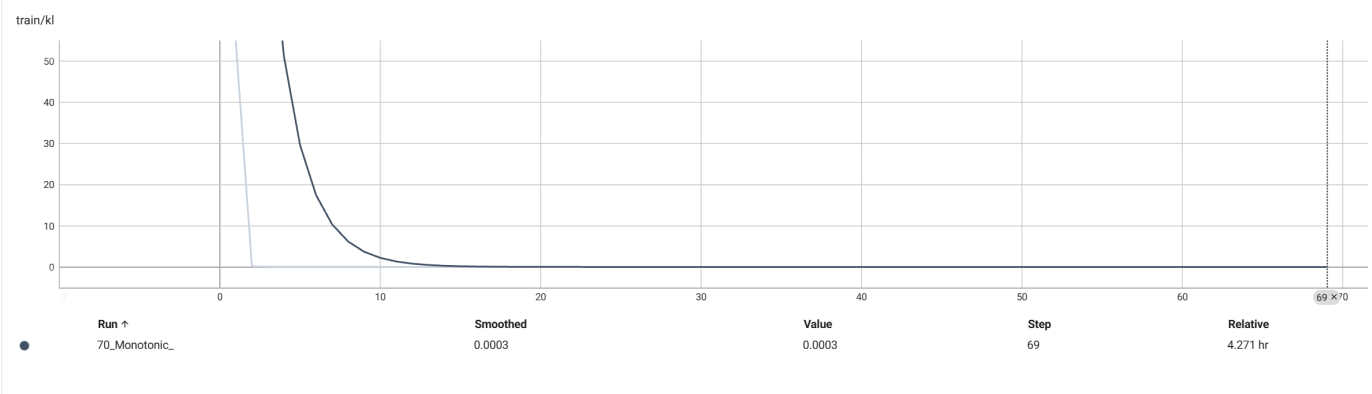


Figure (6) — Monotonic KL Loss

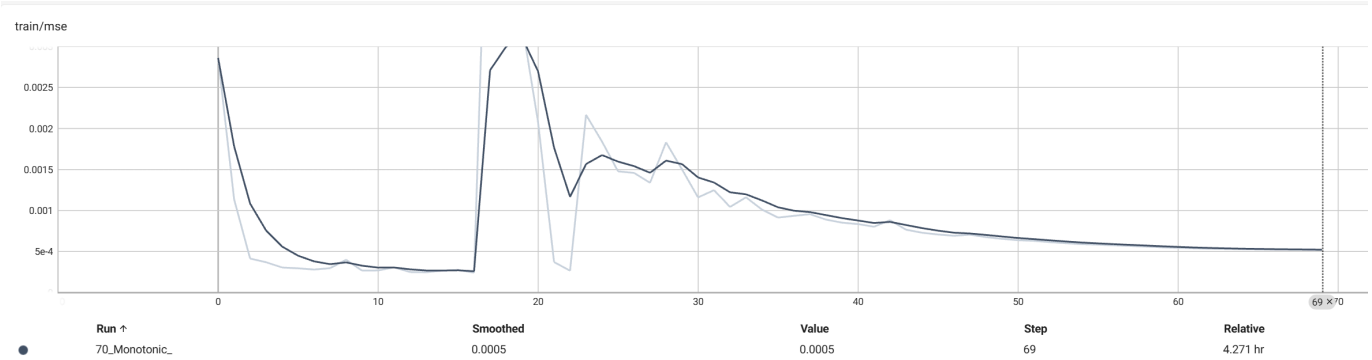


Figure (7) — Monotonic MSE Loss

None

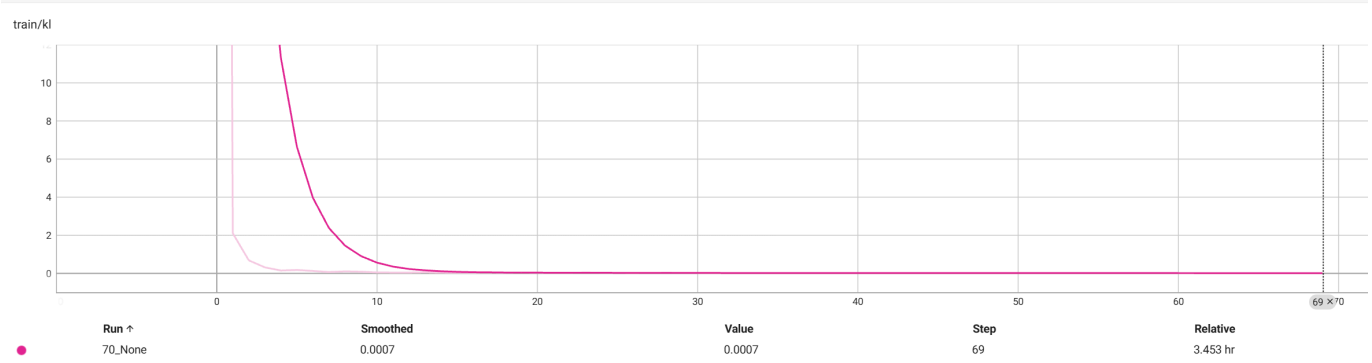


Figure (8) — None KL Loss

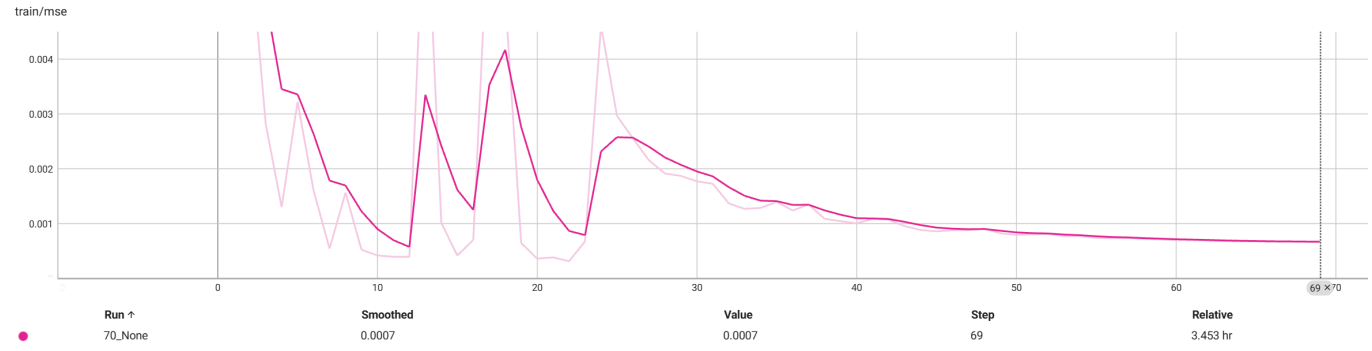


Figure (9) — None MSE Loss

Conclusion

Cyclical 的 KL Loss 初始峰值最高（約 180），MSE 在 Epoch 10 後出現 Spike（峰值約 0.0016），之後有週期性小幅振盪，最終收斂至約 **0.0006**。Monotonic 的 KL Loss 峰值約 55，MSE 在 Epoch 15–20 出現最大的單次 Spike（峰值約 0.003），之後快速下降，最終收斂至約 **0.0005**，為三者最低。None 的 KL Loss 峰值僅約 11，但 MSE 全程震盪最劇烈，多次衝至 0.004，最終收斂至約 **0.0007**，三者中最不穩定。

---

iii. Plot the PSNR-per-Frame Diagram in Validation Dataset (5%)

Hyperparameters

參數	值
Epoch	70
KL Annealing Mode	依曲線各異
Optimizer	Adam
Batch Size	4
Learning Rate	1e-4
KL Anneal Cycle	20
KL Anneal Ratio	0.5
Teacher Forcing Ratio（初始）	1.0
Teacher Forcing 開始衰減	Epoch 10
Teacher Forcing Ratio（衰減）	0.1

Cyclical

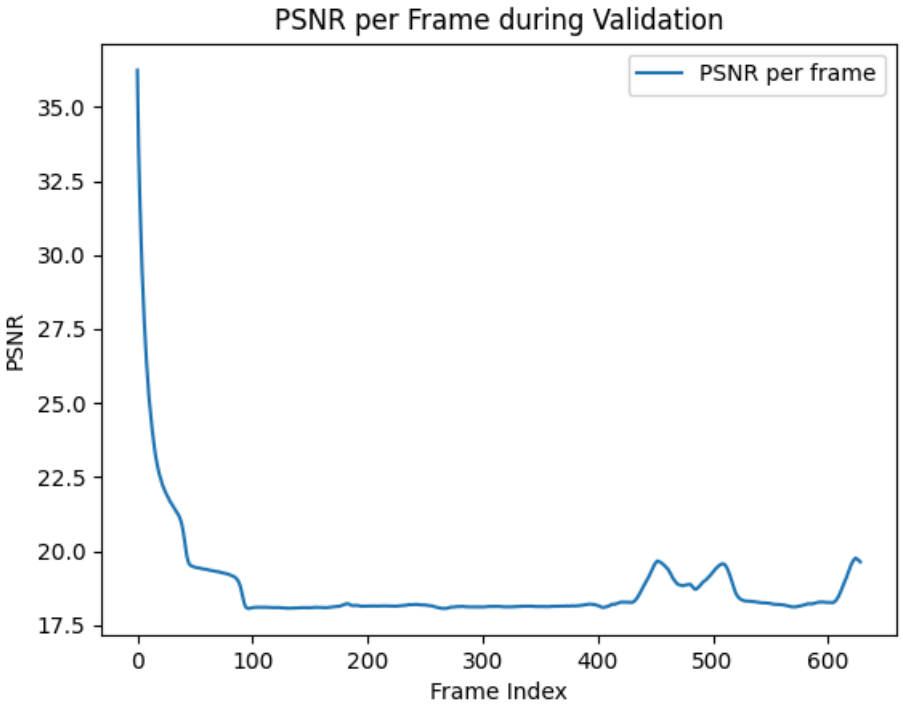


Figure (10) — Cyclical PSNR per Frame

Monotonic

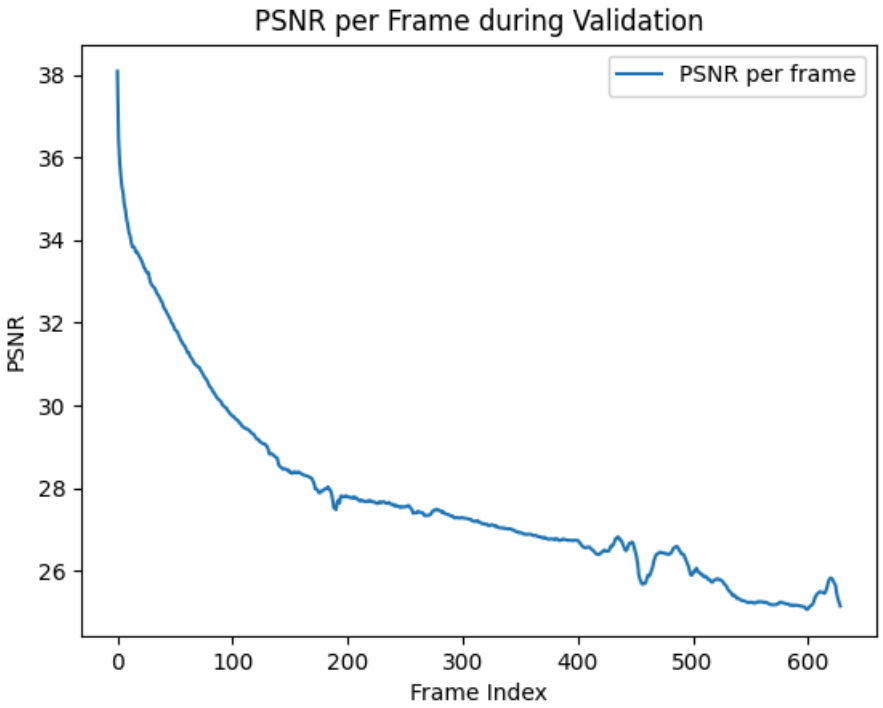


Figure (11) — Monotonic PSNR per Frame

None

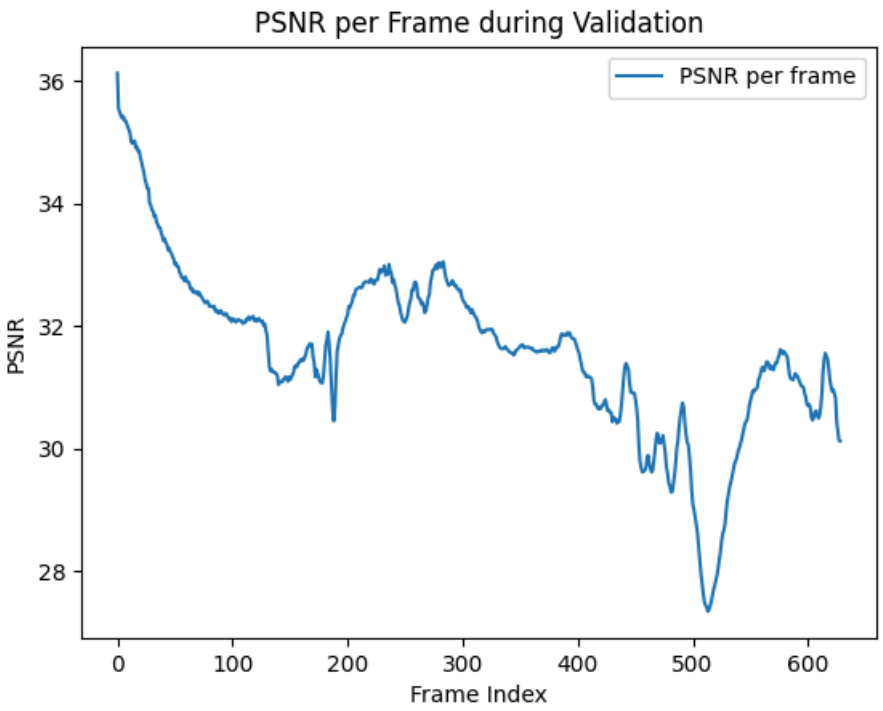


Figure (12) — None PSNR per Frame

Conclusion

Cyclical 的 PSNR 在 Frame 100 後急跌至約 17–20 dB 並幾乎停滯，為三者最差；Monotonic 從首幀約 38 dB 平滑遞降至尾段 25–27 dB，下降最為穩定；None 整體維持在 28–33 dB 最高，但在 Frame 500 附近出現約 27 dB 的低谷，波動較明顯。整體排序為 **None ≥ Monotonic > Cyclical**。

iv. Other Training Strategy Analysis (Bonus 10%)

Different Teacher Forcing Strategy

Hyperparameters

參數	值
Epoch	200
KL Annealing Mode	Cyclical
Optimizer	Adam
Batch Size	4
Learning Rate	1e-4
KL Anneal Cycle	20
KL Anneal Ratio	0.5
Teacher Forcing Ratio（初始）	1.0

Strategy

曲線	設定	說明
綠線 `10_0.1`	保護期 10 圈，快速衰減	較早開始自行生成
橘線 `15_0.05`	保護期 15 圈，緩慢衰減	中速放手
紫線 `30_0.05`	保護期 30 圈，緩慢衰減	最長依賴期

Conclusion

由 PSNR 曲線觀察：在影片預測中，給予較長的 Teacher Forcing 保護期反而會讓 PSNR 降低。以紫線為例，模型在中後期須自行生成影像時難以適應，最終僅收斂至 PSNR ≈ 33.7。

Figure (13) — Different TFR Strategy val/PSNR（200 Epochs）

Defferent epochs

Hyperparameters

參數	值
Function	Cyclical
Optimizer	Adam
Batch Size	4
Learning Rate	1e-4
KL Anneal Cycle	20
KL Anneal Ratio	0.5
Teacher Forcing Ratio（初始）	1.0
Teacher Forcing 開始衰減	epoch=10
Teacher Forcing	0.1

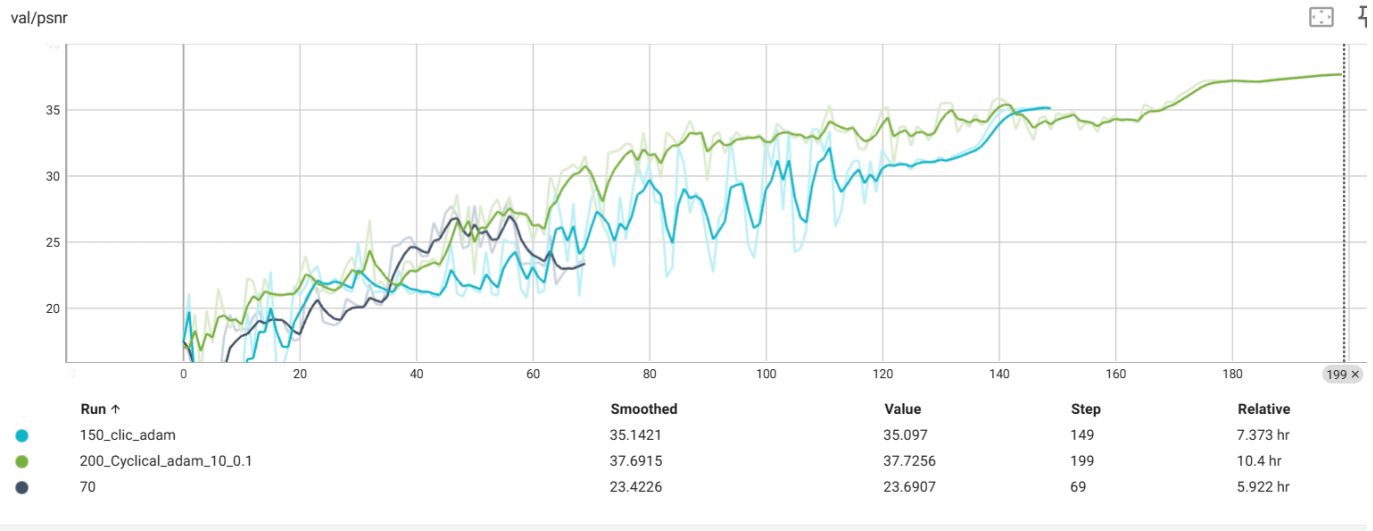


Figure (14) — Different Epochs val/PSNR Comparison

Conclusion

由 val/PSNR 曲線可以明顯看出，訓練 Epoch 數對模型最終效能有顯著影響。70 Epoch（深藍線）收斂過早，PSNR 僅達約 **23.7 dB**，仍處於低平台期；150 Epoch（藍線）呈現穩定上升趨勢，最終收斂至約 **35.1 dB**；200 Epoch（綠線）則持續爬升，在 Epoch 199 時達到 **37.7 dB**，且曲線仍有上升趨勢，顯示模型尚未完全飽和，繼續訓練仍有提升空間。整體排序為 **200 Epoch ≥ 150 Epoch >> 70 Epoch**，說明在此任務中給予足夠的訓練時間是提升 PSNR 的關鍵因素。